Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3714393.3726511

RESEARCH-ARTICLE

Exploiting HDMI and USB Ports for GPU Side-Channel Insights

SAYED ERFAN AREFIN, University of Dayton, Dayton, OH, United States

ABDUL SERWADDA, Texas Tech University, Lubbock, TX, United States

Open Access Support provided by:

University of Dayton

Texas Tech University

PDF Download
3714393.3726511.pdf
18 March 2026
Total Citations: 0
Total Downloads: 210

Published: 04 June 2025

Citation in BibTeX format

CODASPY '25: Fifteenth ACM Conference on Data and Application Security and Privacy
June 4 - 6, 2025
PA, Pittsburgh, USA

Conference Sponsors:
SIGSAC

CODASPY '25: Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (June 2024)
https://doi.org/10.1145/3714393.3726511
ISBN: 9798400714764

Exploiting HDMI and USB Ports for GPU Side-Channel Insights

Sayed Erfan Arefin
sarefin1@udayton.edu
University of Dayton
Dayton, Ohio, USA

Abdul Serwadda
abdul.serwadda@ttu.edu
Texas Tech University
Lubbock, Texas, USA

Abstract

Modern computers rely on USB and HDMI ports for connecting external peripherals and display devices. Despite their built-in security measures, these ports remain susceptible to passive power-based side-channel attacks. This paper presents a new class of attacks that exploit power consumption patterns at these ports to infer GPU activities. We develop a custom device that plugs into these ports and demonstrates that its high-resolution power measurements can drive successful inferences about GPU processes, such as neural network computations and video rendering. The ubiquitous presence of USB and HDMI ports allows for discreet placement of the device, and its non-interference with data channels ensures that no security alerts are triggered. Our findings underscore the need to reevaluate and strengthen the current generation of HDMI and USB port security defenses.

CCS Concepts

• **Security and privacy → Side-channel analysis and countermeasures**; *Software reverse engineering*.

Keywords

Side-channel attacks, Power analysis, I/O ports, USB, HDMI, GPU activity inference, Hardware security, Passive monitoring, Port-based attacks

ACM Reference Format:
Sayed Erfan Arefin and Abdul Serwadda. 2025. Exploiting HDMI and USB Ports for GPU Side-Channel Insights. In *Proceedings of the Fifteenth ACM Conference on Data and Application Security and Privacy (CODASPY '25), June 4–6, 2025, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3714393.3726511

1 Introduction

Today's computers are equipped with numerous Universal Serial Bus (USB) and High-Definition Multimedia Interface (HDMI) ports to support the wide range of external peripherals and visualization needs of modern users. These ports provide a physical channel to the underlying hardware and software, making them integral to the computer's threat surface. As such, they are built with security features designed to prevent exploitation. For instance, USB ports disable the autorun feature to prevent malware on a USB-connected device from executing automatically. Operating systems also notify users whenever a USB-connected device attempts to access the file system, allowing users to block unexpected actions [16]. HDMI ports, primarily output devices, do not support writing to the computer, further reducing their susceptibility to certain types of attacks. These and other security measures help safeguard these physical ports from becoming vectors for attacks [27] [36].

In this paper we argue that current port security defenses are inadequate in the wake of passive attacks that leverage these ports as a vehicle to drive power measurements designed to infer key underlying processes on the computer. Using the example of processes running on the GPU, we demonstrate the viability of these attacks through the development of a custom device that plugs into USB and HDMI ports, facilitating inference attacks on GPU activities.

Our approach is stealthy and does not interfere with the processes that typically trigger user alerts. For the USB port, our attack is designed to avoid manipulating the data channel, thereby bypassing security notifications. For the HDMI port, our device mimics a conventional display device, ensuring it does not trigger any errors or alerts. The ubiquitous presence of USB and HDMI ports on contemporary computers – located at the back, front, and sides – facilitates the discreet deployment of our attack devices. A small, inconspicuous device plugged into any of these ports can remain unnoticed, providing an effective means of conducting attacks without installing software on the victim's computer or attaching hardware to internal components.

The overarching idea behind our attack is as follows: despite modern computers being equipped with over-provisioned power supplies and sophisticated stabilization mechanisms to prevent power instability, significant power draw increases by a GPU during intensive processes can induce detectable fluctuations in power seen by peripheral devices attached to USB or HDMI ports. These fluctuations, though transient and eventually stabilized, contain discernible information sufficient to infer GPU operations. Using GPU loads in the form of matrix multiplications, video scene rendering and neural network execution, we demonstrate that this line of attack poses a potent side-channel that can leak sensitive information about processes running on the GPU.

1.1 Paper Contributions

Our research makes the following Contributions:

(1) GPU power side-channel inference via HDMI and USB ports: Our research introduces a novel family of power side-channel attacks, which utilize power consumption patterns observed at USB and HDMI ports to infer computational processes occurring on a GPU. This method is distinct from traditional attacks that measure power directly from the GPU, as it exploits peripheral connections that either interface with or share a power source with the GPU. Requiring only the insertion of a device into a peripheral port, this

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CODASPY '25, Pittsburgh, PA, USA.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1476-4/2025/06
https://doi.org/10.1145/3714393.3726511

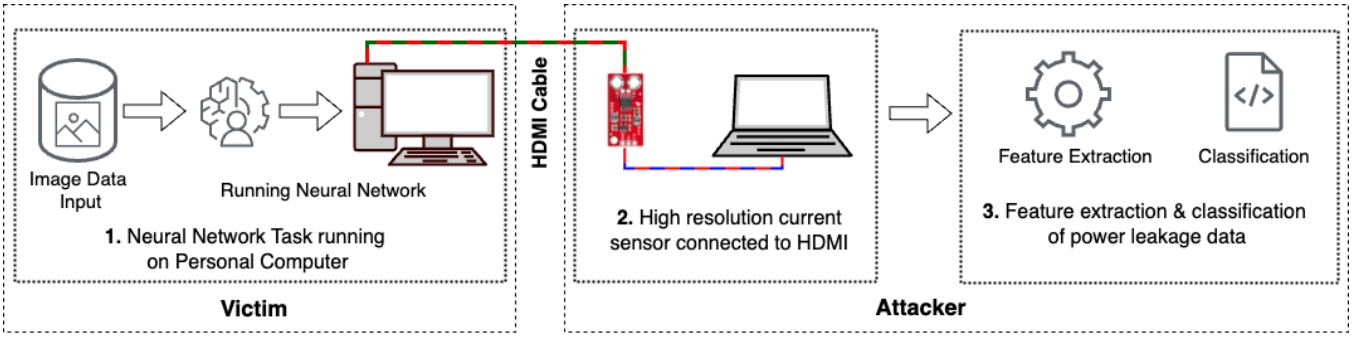


Figure 1: Illustration of the attack process. The example used here is that of a neural network running on the victim computer

stealthy approach operates at the user's privilege level without triggering any alerts. This technique not only broadens the scope of implementable side-channel attacks but also suggests the potential for using peripheral power measurements defensively, such as detecting unusual power draw signatures that indicate malicious activities.

(2) Design of power measurement prototype: We present the design of the power measurement tool central to our attack strategy. At the core of this tool is a SparkFun ACS723 Current Sensor, which utilizes a Hall effect sensor to produce a voltage output correlating to the current through its measurement pins. This sensor is integrated with an Arduino board equipped with a 32-bit ARM Cortex M0+ processor, enabling data collection at a rate of up to 350,000 samples per second. This high-resolution data is crucial for driving the machine learning classifiers that discern and classify GPU activities based on power fluctuations.

(3) Evaluation of attack performance: Our evaluation of the attack's performance begins with an intuitive analysis of visual patterns in time-series data extracted from GPU computations. This preliminary analysis provides initial indications for the feasibility of the attack, leading to the development of machine learning models capable of executing two distinct attack scenarios with high accuracy. In the first scenario, our side-channel approach successfully identifies the specific family of deep neural networks running on the GPU. In the second, it accurately infers which video clips are being rendered by the GPU. These examples demonstrate the broader applicability and effectiveness of our side-channel techniques in targeting GPU-based processes.

1.2 Threat Model

Our threat model assumes an attacker who has physical access to a target computer but lacks login credentials. Such individuals might include office cleaners, maintenance staff, or rogue employees who can access the machine without raising suspicion. The attacker can discretely insert a small power measurement device into an inconspicuous USB or HDMI port on the computer, preferably one that is not easily visible (e.g., the ports at the back). This device is designed to operate stealthily, drawing power data without causing any disruptions or triggering the alerts typically associated with peripheral connections like USB storage devices. The device does not interfere with normal computer operations, such as display functions when connected to an HDMI port.

The attacker's goals may vary from stealing power measurement data linked to high-value algorithms, such as neural network architectures or cryptographic operations, to general surveillance of the user's GPU-dependent activities, which might include video rendering or website browsing. To facilitate the machine learning aspect of the attacks, the attacker would need to acquire training data that reflects the specific computational processes targeted. In our laboratory setup, we utilized a wired connection for data transfer, sufficient for demonstrating the attack's foundational concepts. However, in a practical application, this setup could be easily modified to include wireless data transfer, such as using Bluetooth or Wifi to transmit data to a device in a nearby location. Figure 1 captures the attack process, taking the example where the attacker seeks to infer information about a neural network architecture running on the victim's device.

1.3 HDMI/USB Power Side-Channel for Defense

While the primary focus of our paper is on exploiting the described side-channel for attacks, it is important to note its potential defensive applications. Specifically, this technique could be useful for anomaly detection or malware identification through deviations in power consumption patterns. For example, a computer infected with malware that engages in resource-intensive activities unknown to the user — such as crypto-mining, data exfiltration, or unauthorized network attacks — would exhibit abnormal power usage. Employing a USB-type or HDMI-type power measurement device can provide a significant advantage in such scenarios. It operates independently of the computer's software, making it less susceptible to tampering by malware that might disable or manipulate software-based monitoring tools. Furthermore, its simple installation and ease of use make it accessible to users without specialized technical skills, offering a non-invasive, reliable method for monitoring system integrity.

2 Related Works

Our research has two clusters of related work: power side-channel attacks on computing devices, and attacks that exploit vulnerabilities of I/O devices. In this section, we discuss these two families of research and describe how we differ from them.

2.1 Power side-channel attacks:

The wide array of past studies on power side-channel attacks measured power in several different ways depending on the threat scenario and targets of the attack. We categorize this related work based on the ways in which power was measured before discussing how our work differs from this body of past research.

Using Nvidia-smi interface: Nvidia-smi is a widely used interface to measure nvidia GPU resources, including power consumption [3]. The interface utilizes the nvml library [2] provided by Nvidia GPUs. The research conducted by Jha et al. [23] investigates a two-stage attack methodology designed to infer the architecture of deep neural networks (DNN). The study utilizes nvidia-smi to analyze performance on P100 and P4000 GPUs, demonstrating the attack's effectiveness. The authors proposed a secure MobileNet-V1 as a mitigation to such vulnerability. In their work Sabbagh et al. [38] presents an overdrive fault attack on modern GPUs, exploiting nvidia-smi to introduce random faults during kernel execution. The authors effectively recovered AES keys in their experiments. The study by Sayed et al. [10] explores power consumption patterns using nvidia-smi to determine if this data could reveal key details about DNN architectures running on GPUs. They conducted model inference attack on several nvidia GPUs including some cloud GPUs.

Using Intel's Running Average Power Limit (RAPL) interface: RAPL is the primary interface for power measurement on Intel CPUs. Moritz et al. [28] introduced a software-based power side-channel attack that leverages unprivileged access to this interface. The authors successfully extracted cryptographic keys and monitored application control flows without needing physical access or specialized hardware. Demonstrations included breaching Intel SGX to extract AES-NI keys, and recovering RSA keys via privileged attacks. Xiang et al., [47] conducted a research where they introduced a novel approach to model extraction in deep learning using the Intel Running Average Power Limit (RAPL) to exploit power leakage in ReLU activation functions via a software interface. Their attack used a very small number of queries to extract deep learning models by leveraging software-based power side-channel information. This technique was implemented on the oneDNN framework. They extracted a 5-layer MLP and Lenet-5 CNN. The study conducted by Segev et al., [40] showed how performance counters can be used to infer information about neural networks. They used the Intel Power Gadget to collect detailed performance traces, including power consumption and CPU utilization. The software used the Intel RAPL technology in its core.

Exploiting Dynamic Voltage and Frequency Scaling (DVFS): DVFS is a power management mechanism used in Intel, AMD, and ARM CPUs, that dynamically adjusts CPU frequency and voltage to reduce overall system power consumption and enhance performance per Watt. While DVFS helps in managing energy efficiency, it also introduces a potential vulnerability that can be exploited through power side-channel attacks. For example, by observing the power consumption variations that occur as a result of DVFS adjustments during the computation process. Chen et al., [29] successfully extracted AES keys by leveraging the DVFS mechanism, while Yingchen et al., [45] studied how power side-channel attacks on modern Intel and AMD x86 CPUs can be used via Dynamic Voltage and Frequency Scaling (DVFS), without requiring access to a power measurement interface directly. Debopriya et al., [15] used the DVFS side-channel attack to perform website fingerprinting for browsers like Google Chrome and Tor across various CPU architectures, while Sehatbakhsh [41] manipulated the power management units to enable side-channel attacks through electromagnetic emanations from the voltage regulator module (VRM). They utilized this technique to execute keystroke logging.

Direct power measurement using an oscilloscope: Hasindu et al., [17] outlines a power analysis attack, where they focus on extracting cryptographic keys from devices by analyzing power consumption data during cryptographic operations. The researchers demonstrate direct power measurement using an oscilloscope. They utilize power analysis techniques to break the Speck algorithm for embedded systems. Luo et al., [30] study the power side-channel on Graphics Processing Units (GPUs) to extract a secret key from a block cipher operating on a GPU. In order to capture power they inserted a resistor in series with the PCI-Express power supply and measured the power physically. They targeted a CUDA AES implementation on an Nvidia Tesla GPU.

Phone charging stations: Yang et al., [46] highlighted the privacy risks posed by public USB smartphone charging stations whose public placement makes them vulnerable to malicious entities that rigg them with power measurement functionality. Using one such rigged hub, the researchers demonstrated a side-channel attack that can identify web pages loaded on a smartphone during the phone charging process. They conducted their research with different browser settings, network types, and battery levels, revealing successful attacks under all these settings. Sraddhanjali et al., [7] extended this line of attack and demonstrated that USB charging hubs can analyze power consumption data to infer YouTube music videos that a user is watching on their smartphone, while Alexander et al., [26] focused on the wireless charging interfaces on modern smartphones. They showed that the wireless charging interface is also susceptible to power-side-channel attacks, specifically demonstrating a website fingerprinting attack on iOS and Android devices.

Other power measurement approaches: Zhao et al., [48] showcased a new security vulnerability introduced by integrating Field Programmable Gate Arrays (FPGAs) into cloud data centers and System-on-Chips (SoCs). Their work reveals that FPGAs can facilitate software-based power side-channel attacks remotely without the need for physical access to the target system. The researchers demonstrate using ring oscillators on FPGAs to monitor power consumption of both FPGA modules and CPU components within the same SoC, successfully executing power analysis attacks on RSA cryptomodules and bypassing timing-channel protections for CPUs. Sayed et al. [9] demonstrated a power side-channel attack on the GPU to infer deep neural network architectures using a software-based approach. Their attack reads power consumption data from the on-board sensors of the GPU.

How we differ from these works: The fundamental difference between these studies and our research is that we introduce a novel side-channel – specifically, power leakage through peripheral devices such as the HDMI and USB ports. While previous work have relied on either invasive physical measurements directly from the power supply or sophisticated software approaches that require

specific system access or vulnerabilities, our approach work by methodically triggering a power draw from the HDMI/USB ports that then enables inferences on the underlying processes running on the GPU. This is achieved without altering the device or needing special privileges. Notably, our method capitalizes on the inherent power variations that pass through these peripheral ports as a byproduct of GPU processing loads, making it a unique point of observation that remains largely under-explored in current literature. Our technique, therefore, opens new avenues for both understanding and mitigating security risks in modern computing environments where external peripherals are common.

On the surface, the charging station attacks (e.g., [7, 26, 46] might seem to have some similarity to our work given that they also use a USB port. However, that attack is fundamentally different from our attack since the target of the attack is the phone plugged into the USB port, and the power being drawn by the phone is directly measured by a circuit placed between the phone and the USB port. Inferences about activities being undertaken on the phone are then done based on these direct power measurements. In our attacks on the other hand, the USB (and HDMI) ports are simply a subset of the many peripherals that share a power source with the GPU, indirectly picking up GPU power patterns through fluctuations seen in its own power as supplied by the source. These indirect power patterns are then used to infer information about computations that occur on the GPU.

2.2 I/O port related vulnerabilities:

The body of past research on I/O security vulnerabilities related to our research is mainly focused on the USB port, given its usage for a wide range of device connectivity applications. We only briefly review these works, given that they do not focus on the power side-channel which drives our work.

Ramadhanty et al. [37] investigate the vulnerability of Windows 10 to USB-based keylogger attacks. By embedding a PowerShell script in an Arduino Micro, they demonstrated a successful keyboard injection attack. Pham et al. [35] examine the information security risks introduced by USB storage devices due to their insecure design and open standards. Their work reviews various USB-based software attacks on host computers and USB devices, exploiting vulnerabilities in USB protocols, embedded security software, drivers, and Windows Autoplay features.

Nohl et al. [32] investigate an attack in which USB firmware is reprogrammed to bypass traditional security measures and execute unauthorized commands, while Jeong et al. [22] analyzes the vulnerabilities of popular secure USB flash drives. They found that passwords can be exposed during communication between the USB drive and the PC. Furthermore, the study reveals that some secure USB flash drives are susceptible to firmware attacks, allowing unauthorized users to bypass security measures.

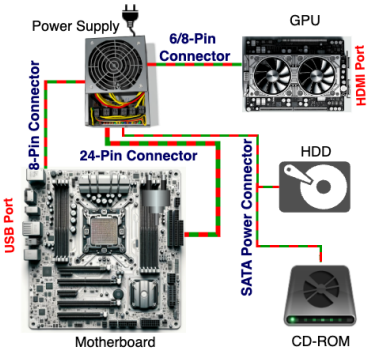
Brian et al. [8] show how the USB Switchblade could be used to steal sensitive information such as passwords, network configurations, and system details, making it a powerful tool for attackers to gain unauthorized access and steal data from a target machine. Kaspersky Lab [24] discusses scenarios in which a compromised USB storage device can be used to secretly extract sensitive data from a victim machine. This process occurs without the awareness of the host or the USB device owner.

Tariq et al. [43] review cybersecurity challenges in IoT, focusing on vulnerabilities in USB-connected devices. They discuss the risks of malware spreading through USB ports and emphasize the need for securing these interfaces. The authors highlight the importance of developing robust security measures to mitigate these threats, suggesting the use of better authentication protocols and noting that traditional security measures are inadequate for modern IoT environments.

How we differ from these works: While previous work share our objective of highlighting the security threats posed by USB devices, our approach is distinct in that it focuses on a passive power measurement attack. Unlike these prior attacks, our method does not require any special privileges or engage in any communication with the underlying operating system or applications. Instead, it passively measures power consumption patterns, making it a stealthier and less intrusive method compared to attacks that actively probe the target system.

3 Basics of Computer Power Supply Mechanisms

To provide context to our attack design, this section covers some basic information about the power supply mechanism is computers. We first discuss the power supply unit (PSU) and how power is supplied to different peripherals of the computer. We finally discuss some input and output ports (I/O ports) that can potentially power external devices.



(a) How power is supplied inside the computer components.

(b) A sample ATX Power Supply

Figure 2: Desktop Computer Power supply.

3.1 Computer Power Supply Unit (PSU)

Often the power supply used in a home-usage, gaming computer or a workstation is an ATX power supply. A sample picture of an

ATX power supply is shown in Figure 2b [39]. Various power cables are used to connect all the components to the power supply. Each type of connector has specific current and voltage ratings. A very widely used connector is the 24-pin ATX power connector, which connects to the motherboard. It can deliver multiple voltages, such as +3.3V, +5V, and +12V. The 24-pin connector can supply significant current, up to 24A. However, this current supply depends on the power supply's capacity. Most computers are equipped with high-performance graphics cards. These are used for advanced graphics processing and machine learning tasks. The 6-pin or 8-pin PCIe connector can deliver high power to GPUs [5]. An illustration of the power supply connectors and the corresponding computer components is presented in Figure 2.

3.2 Computer I/O ports

There are several input and output (I / O) ports in the computer that can be used to connect different peripherals. These peripherals can be a wide variety of devices. In case of USB devices, keyboard, mouse, external hard disk drive, etc. are a few notable peripherals. These devices require power to operate, which is consumed over the USB port. The current flow from a regular USB port can vary depending on the specific USB standard and the device providing the power. For USB 2.0, the standard current output is typically 500 mA (milliamperes) at 5 volts, providing a maximum power of 2.5 watts. For USB 3.0 and later versions, the current output can be higher, often up to 900 mA or more, again at 5 volts. In our experiments, we used the USB 2.0 ports.

We also observed the current draw made by the HDMI port. The HDMI port is in several ways a more complicated port than the USB port. It does not supply active power to the display devices, as those require external power. However, it has a power saving option, where the device goes to sleep if there is no signal in the HDMI port. This is called hot plug detection (HPD). There is a small circuit that periodically checks for the signal such that, when there is a signal, the display device can come up from its sleep status. In order to run this small circuit, a small amount of power is supplied over the HDMI port. Pin 18 of the port supplies 5v and at most 500 milliampere, to run this circuit. In our experiments described in Section 5 we monitor the 18th pin of the HDMI port and the power pin of the USB port. Details of the sensor and circuit used to make measurements on this pin are discussed in Section 4.1.

4 Attack Design

In this section of the paper, we discuss our hardware setup to measure current and possible power leakage from the I/O ports. The following sections describe the circuits used and the hardware setup in Section 4.1. The computer used to run our experiments is described in Section 4.2.

4.1 Hardware setup

At the heart of our hardware lies a current sensor (SparkFun ACS723 [42]) capable of recording the current passing through a series circuit by using a hall effect sensor, which ensures electrical isolation between the monitored circuit and the measuring circuit. The module can measure a current from 10mA to 5A. The analog output of the module is set to 20kHz in order to reduce noise as the gains are set to high. The sensor is connected to an Arduino to read the analog signal and produce a digital reading. In our experiments, we used an Arduino MKR Zero board which uses a 32-bit ARM Cortex M0+ processor [1] that can achieve 350 KHz analog-to-digital converter sampling rate. To avoid triggering the short-circuit protection of the I/O boards we put a load in the series while measuring current. Power dissipation generates heat, so distributing it across multiple resistors helps prevent overheating. In our experiment, two parallel 20Ω film resistors (R3 and R4) were used, as shown in Figure 3. The final assembly of our attack device is presented in figure 4 which labels the components of the device. The Arduino's ADC was configured at 12-bit resolution to continuously sample at 40 kHz, exceeding our 20 kHz requirement. Using interrupts and an ISR, we handle conversion results.

In Section 1.2, we discussed the possibility of enhancing the attack device by integrating a Bluetooth or WiFi module, thereby extending the operational range of the attacker. The attack device, in this extended setup, would draw power from an independent source, such as a 5V battery.

4.2 Experiment Machine

The experiments were carried out on a desktop computer that had the following configuration. The graphics processing unit (GPU) used in the computer is an Nvidia RTX 2060 Super with 8GB graphics memory. This uses the Nvidia Turing Architecture. The other components of the computer include a Central Processing Unit (CPU) Intel Core i7 9600K Processor, 16 GB RAM, 4 TB hard drive (HDD). The operating system (OS) used is Ubuntu 22.04 LTS. The tensorflow version used in the experiment is a tensorflow-gpu 2.12. The computability of the tensorflow setup is 7.5. The power supply used in the computer is a 450 watt power supply. The GPU uses a 6-pin power connector. GPU requires a 175 Watt power and CPU requires a 95 Watt power. The other components draw a variable power based on the need.

5 Attack Experiments

In this section, we present our experimental methodology. We begin with preliminary experiments demonstrating power leakage (Section 5.1), then describe data collection and configurations for Convolutional Neural Networks model architecture inference (Section 5.2.1) and for video inference from rendering tasks (Section 5.2.2). Finally, we detail the datasets used in the CNN and video inference experiments (Sections 5.3.1 and 5.3.2).

5.1 Preliminary Experiments

Before undertaking a detailed design of the attack, we first conducted a preliminary experiment to discern if GPU operations produce measurable current draw fluctuations at our attack device. In the first of these preliminary attacks, we used the Glmark 2 software [18] to render videos on the GPU. This software spawns a window and renders scenes that make use of OpenGL (ES) 2.0 and is commonly used to benchmark GPU performance. Before collecting any data, we ran the command to see GPU activity using the Nvtop tool [33] and observed some GPU utilization. Figure 6 captures the findings from this experiment for one of the scenes. The figure reveals a slight hump in between the 1 and 2-second

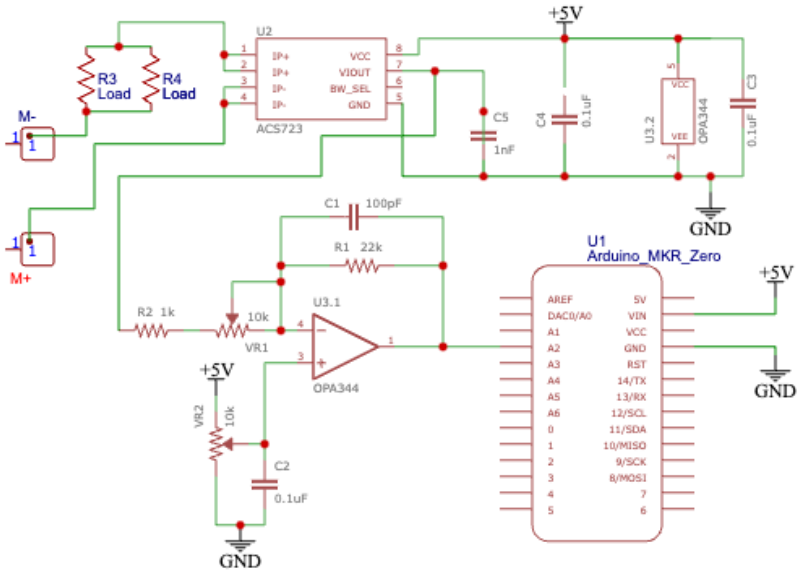


Figure 3: Circuit of the attack device.

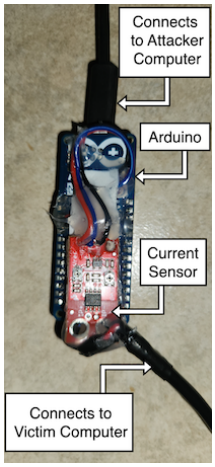
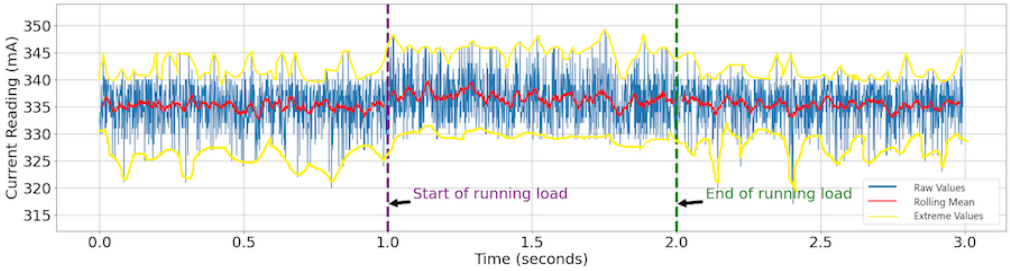
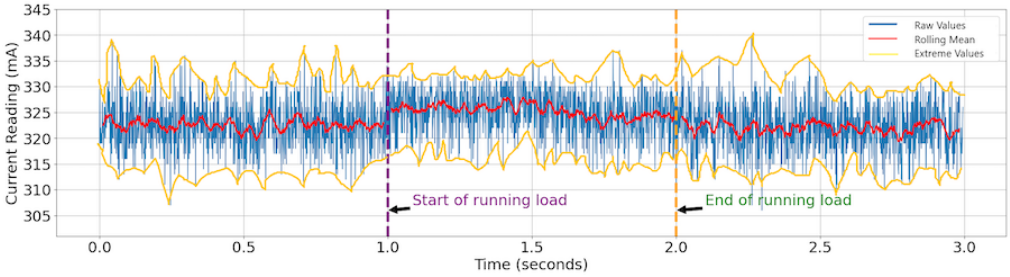


Figure 4: The attack device.



(a) HDMI



(b) USB

Figure 5: Raw reading for Scene rendering with Glmark software.

marks when the rendering happens. For both the USB and HDMI ports, our device is hence able to pick up the power fluctuations triggered by the video rendering on the GPU. Similar patterns (not shown here) were observed for various other scenes, providing us with the first signs that our line of attack might be feasible.

Our second preliminary experiment focused on matrix multiplication, a fundamental computation underlying many GPU operations such as image processing and neural network computations. We aimed to observe power leakage from the I/O ports during different types of matrix multiplication. To formulate this experiment, we used matrices of size 1920 x 1080, matching the resolution of a typical full HD display. This setup mimics a regular use case where a matrix of this size is executed on the GPU. The goal was to multiply two matrices on the GPU, ensuring execution via the CuPy library [44], which utilizes the GPU for matrix manipulation operations.

We created four matrices, A, B, C, and D, with random values between 0 and 255 to replicate the color values of each pixel. We then performed two matrix multiplication tasks: A multiplied by B, and C multiplied by D. Each multiplication task was repeated 25

times to observe the power leakage phenomenon. Following the multiplications, we extracted features, specifically the mean and MFCC 2 (Mel-frequency cepstrum coefficient) of the time series data, and plotted these features. This experiment was conducted for both HDMI and USB I/O ports. The feature plots are presented in Figure 6. The plots show that our USB and HDMI measurements depict distinct clusters of power measurement data for the two different matrix multiplication tasks conducted on the GPU. Given these positive results from the two toy experiments, we proceeded to design and execute the fully-fledged attack experiment that we discuss next.

5.2 Attack Configurations and Data collection

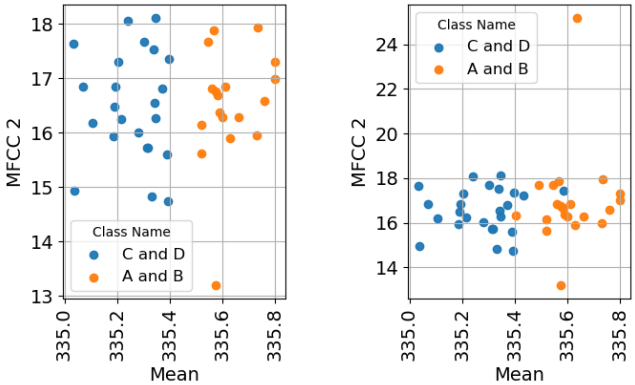
In this part of the research, we focus on the detailed experiments of our research. We first run experiments to infer Convolutional Neural Networks and later infer video rendering.

5.2.1 Inference of CNN Model Architecture running on the GPU. Here we focus on the inference of the architecture of the convolutional neural network (CNN) using the power monitoring from the I/O ports. The CNNs are now widely used in many applications for a wide range of applications such as Image classification, Object detection, Video analysis, etc. The victim runs a CNN to classify the image, and the attacker will monitor the I/O port and try to infer the architecture of the CNN. We conducted this experiment for both HDMI and USB I/O ports. In this meticulously designed experiment, we postulate that the attacker has ingeniously adapted well-established CNN models and has proficiently trained machine learning models using the I/O port power data. Upon acquiring I/O port power data from the victim's machine, the attacker is equipped to precisely classify the specific CNN that was operational on the victim's system. Our investigation focuses primarily on the CNN architectures listed in Table 1, all of which benefit from the availability of pre-trained models in Tensorflow [6].

The experimental procedure begins with vigilant I/O port monitoring, followed by the beginning of CNN classification using the image data sets delineated in 5.3. To ensure meticulous data integrity, a deliberate interval of 10 seconds is observed between each data collection sample. Recognizing the potential utilization of alternative CNN models by the victim, our experimental framework is also equipped to detect the presence of any unidentified model classes. We meticulously crafted our experimental design into two distinct configurations, Configuration 1 and Configuration 2, to probe the depths of CNN architecture inference. In Configuration 1, the attacker is armed with prior knowledge of the target CNN model's power consumption patterns, which are included in his training dataset. This setup allows for a direct and informed inference of the CNN model. Moving to Configuration 2, we introduced a layer of complexity where the attacker is only aware of the power data for one variant of the CNN model. This strategic limitation challenges the attacker to deduce the specific family to which an unknown variant of the CNN model belongs, based on partial data. This approach enhances our understanding of how well an attacker can generalize from known models to detect novel yet related model architectures. The data sets used in this experiment, which serve as input to CNN, are thoroughly described in Section 5.3.1.

Base Name	Variants
ConvNeXt	ConvNeXtBase, ConvNeXtLarge, ConvNeXtSmall
VGG Net	VGG16, VGG19
ResNet	ResNet50, ResNet152V2, ResNetRS420
InceptionNet	InceptionV3, InceptionResNetV2
MobileNet	MobileNet, MobileNetV2
DenseNet	DenseNet121, DenseNet169, DenseNet201
NASNet	NASNetMobile, NASNetLarge
RegNet	RegNetX002, RegNetX160, RegNetY320
EfficientNet	EfficientNetB0, EfficientNetB5, EfficientNetV2L
XceptionNet	(No variants)

Table 1: Base CNNs and their variants used in Experiments



(a) HDMI **(b) USB**

Figure 6: Plots of Mean feature and MFCC 2 for Matrix Multiplication experiments.

5.2.2 Inference of Videos Rendered on the GPU. Now a days, creating videos for the purpose of entertainment, video blogging (vlogging), or education is a very common task. One of the major use cases of the GPU of a computer is rendering videos. We further conduct an experiment evolving around video rendering. We design this experiment where the attacker wants to identify some video which is being rendered on the victim's machine. To run the experiment programmatically, we utilized the OpenCV library [11] with Python. Here we use 10 different videos from different sources and try to classify the video. We assume that the attacker can train machine learning models with power leakage data for different videos. If such a video is rendered on the victim machine, the attacker can identify that by observing the power leakage data over the HDMI port of the victim machine. In this experiment, each video is cut for 10 seconds after the video is played for 10 seconds. We repeat this process for all the videos and 25 times each.

5.3 Experiment Datasets

In the following section, we present a detailed overview of the datasets utilized in our experiments. We describe the characteristics of these datasets, emphasizing their relevance for our experimental objectives.

5.3.1 Image Datasets for testing CNNs. The Convolutional Neural Networks (CNNs) typically take an image as input to perform different tasks (such as classification). This section discusses the image datasets used in our experiments for the CNNs. The images were taken from the ImageNet dataset [14]. The ImageNet dataset is widely used in the ImageNet competitions, which has produced several CNNs over the years, such as VGG, ResNet, etc. We used 2 different categories of images. Dataset 1 contains the images of 50 cats and 50 dogs. Cats and dogs are typically misclassified among themselves. Dataset-2 contains 100 random images. We have further divided the data sets into two parts. In the later part of our experiment, we classify the CNN architectures – inferred from the collected Power leakage. We had 80 images selected for the train set and 20 images for the test set. There were no common images between the train and the test sets.

5.3.2 Video Inference Experiment. In this experiment we need to use video files as input to video rendering experiment. In the process of determining the appropriate format and quality of the video utilized in the Video Identification Experiment, the ubiquity of smartphones as recording devices was considered. Contemporary smartphones generally possess the capability to record in Full HD (1080p) or superior resolutions. The mp4 file format, commonly employed for the storage of video content, coupled with the H.264 video codec, strikes an optimal balance between superior video quality and minimized bandwidth consumption, rendering it exemplary for streaming and broadcasting applications. Consequently, the H.264 codec was selected for post-editing video rendering. The corpus of video files chosen for this experiment comprised ten videos, each with a duration ranging from a minimum of 30 seconds to a maximum of one minute.

6 Evaluation

This section provides a comprehensive evaluation of our experiments, divided into Preliminary Data Exploration (Section 6.1) and Classification Results (Section 6.2). It highlights the attack's accuracy and the features influencing CNN model architecture inference and video rendering tasks on the GPU via I/O ports (USB and HDMI).

6.1 Preliminary Data exploration

In this study segment, we discuss how the raw data was prepared with feature extraction and removal of empty entries for classification. Initially, the data were captured in analog values form during the experiments. Afterward, each sample was stored as an array and subsequently organized into a dataframe. Missing or NaN values were removed after the data collection. In order to identify all CNN models from the collected power leakage data, we employed statistical and spectral feature extraction methods using Python's NumPy [20] and Librosa [31], respectively. The extracted features, used in our experiments, both statistical and spectral, are outlined in Table 2.

After the feature extraction process, we investigated the features that were extracted. For this part, we considered the features extracted using the Dataset-2 for visualization purposes. In order to identify the best features, we used the Scikit learn's [34] Select K-Best method for the top 30 features. First, we discuss the CNN models identification feature extraction process. We found that the feature sum and roll-off (1st roll-off) are of the highest importance in terms of both statistical and spectral features. It is important to mention that many spectral features produce more than one value for that specific based on the characteristics of the given time series. We also observed that chroma, flatness, SEM, and RMS are of higher importance. We presented the importance value created by the Select K-Best for the top 30 features in Figure 7. In addition, we investigated the values of some of the top features. In Figure 8, we present the two main features - Sum (Statistical feature) and Chroma 9 (Spectral feature). This figure presents box plots of the values for all CNNs involved in the experiment for those two features. We can observe that some of the CNN models are distinguishable visually by observing these two features.

Feature Type	Extracted Features
Statistical Features	Summation, Mean, Mean Absolute Deviation (MAD),Standard Deviation, Variance, Skewness, Standard Error of the Mean (SEM), Kurtosis
Spectral Features	Spectral Centroids (Multiple), Spectral Bandwidths (Multiple), Spectral Flatnesses (Multiple), Spectral Roll-offs (Multiple), Tempograms (Multiple), Spectral Contrasts (Multiple), Fourier Tempograms (Multiple), Zero Crossing Rate, Mel-frequency Cepstral Coefficients (Multiple), Tempo, Chroma (Multiple), Tempogram Ratio, Root Mean Square (RMS)

Table 2: Statistical and Spectral features that were extracted from the collected data.

Afterwards, we discuss the feature extraction outcome of the video identification experiment. We found the top features to be Sum, Chroma 9, Flatness 0, Chroma 11 and Rolloff features. In this part, we visualize the values of two spectral features, Chroma 9 and Flatness 0. The values of these two features are presented in figure 9. It can be seen that Chroma 9 and Flatness 0 have visually distinguishable features for the 10 different videos.

6.2 Classification

In our research, we utilized the Scikit-learn library [34] to develop pipelines using various scalars and classifiers for data preprocessing and classification tasks, respectively. The scalars included: MaxAbs Scaler, ensuring the maximum absolute value of each feature is 1.0 without centering; Min-Max Scaler, adjusting features to the minimum and maximum range; Standard Scaler, normalizing features by removing the mean and scaling to unit variance; Quantile Transformer, mapping data to a uniform distribution and minimizing outlier effects; and Normalizer, scaling samples to unit norm, suitable for datasets with many zeros and varying scales. Classifiers employed were K-Neighbors [19], Light GBM [25], Support Vector Machine (Multiclass) [21], Random Forest [12], and XGBoost [13]. We used the K-best algorithm to select the top 30 features for the pipelines, choosing the classifier with the highest test accuracy and the F1 score. We also set a confidence threshold of 0.5 for all

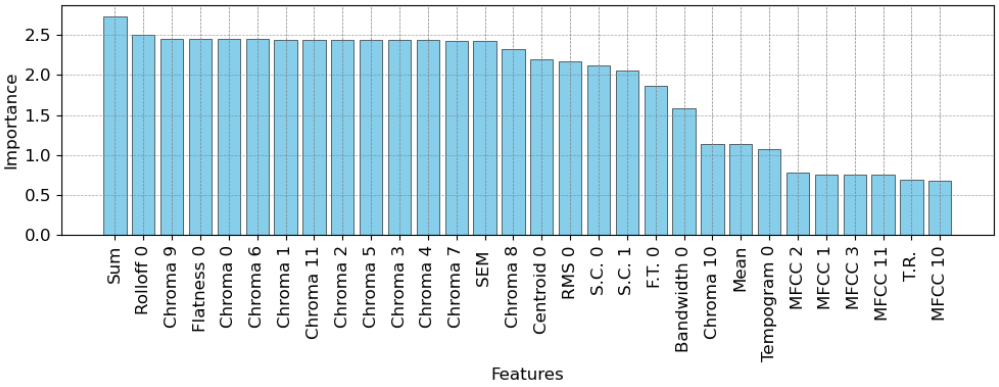
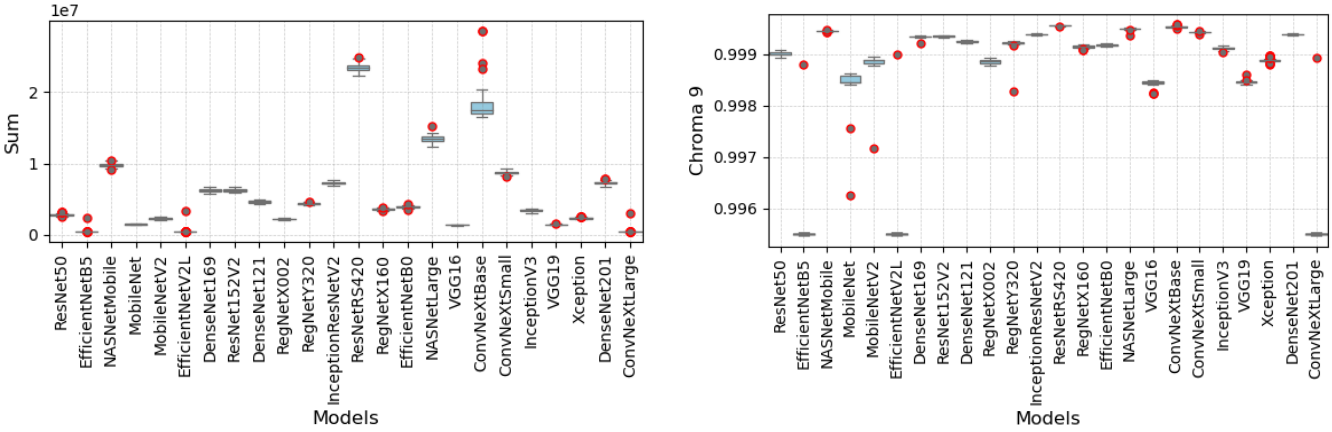
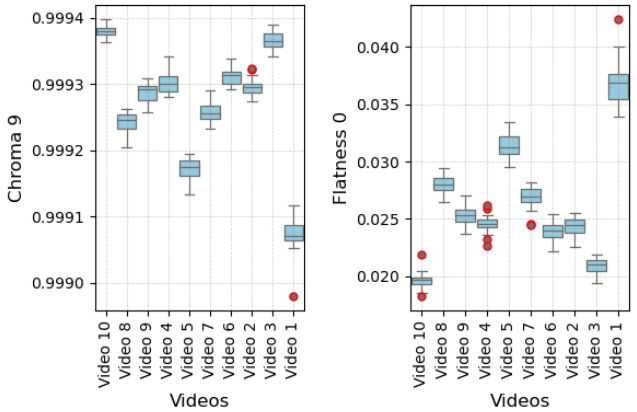


Figure 7: Feature importance from Scikit learn's Select K-Best for the top 30 features, derived from CNN models on Dataset-2. Abbreviations: F.T. (Fourier Tempogram), T.R. (Tempogram Ratio), S.C. (Spectral Contrast).



(a) Feature: Sum **(b) Feature: Chroma 9**

Figure 8: Box plots presenting the top two features - Sum (Statistical feature) and Chroma 9 (Spectral feature) for all the CNN models involved in the experiments. The red dots represents the outliers.



(a) Chroma 9 **(b) Flatness 0**

Figure 9: Plots of Mean feature and MFCC 2 for Matrix Multiplication experiments.

classifiers to handle unknown class cases. Next, we discuss the classification results for the CNN and Video identification experiments in Section 6.2.1 and 6.2.2.

6.2.1 Classification results for CNN model architecture inference experiment. We first identify the base models of the networks mentioned in Table 1 with multiclass classification. As mentioned before, we execute the sckit-learn pipeline composed of the scalar and the classifier. We run the classification step on both data sets (datasets 1 and 2) independently. This is formulated to be a 10-class problem. We conducted the classification task with the possibility of an unknown class and also without the possibility of having an unknown class. The test accuracy and the F1 score for the best performing model are given in Table 3. We can observe that, while monitoring the HDMI port, Dataset-2 had performed well with an accuracy of 94.25% for the regular classification. Considering having an unknown class, the Dataset-2 performed better than the Dataset-1, having a test accuracy of 93.75%. In both cases, the F1 score is higher. Furthermore, we present a confusion matrix for the

Dataset-2 classification in Figure 10. We can observe that in some of the test cases, MobileNet was misclassified as VGG16, InceptionNet was misclassified as MobileNet, and RegNet was misclassified as EfficientNet. Moreover, upon meticulous examination of the USB port, Dataset-2 exhibited a remarkable accuracy of 98.95% when accounting for the potential presence of an unknown class. In contrast, when the scenario excluded the consideration of an unknown class, Dataset-1 demonstrated superior test accuracy, achieving an impressive 71. 35%. Subsequently, we executed the classification for Configuration 2. It should be noted that, during the HDMI port monitoring, Dataset-2 exhibited commendable performance, achieving an accuracy of 76.45% in standard classification scenarios. When incorporating an unknown class into the analysis (by setting the confidence threshold to 0.5 and providing unknown power leakage data), Dataset-2 outperformed Dataset-1, registering a test accuracy of 74.39%. In both cases, the F1 scores were significantly higher, underscoring the robustness of the model. Furthermore, a thorough investigation of the USB port revealed that Dataset-2 maintained a remarkable accuracy of 74.39% while considering the potential inclusion of an unknown class. In contrast, in scenarios where the unknown class was not considered, Dataset-2 showed superior test accuracy, attaining an impressive 72.10%. The results are presented in Table 4. Across all instances examined, the K-Neighbors classifier emerged as the superior model, employing data normalization via the sklearn normalizer scaler. This model was meticulously configured through a pipeline designed to fine-tune hyperparameters. The optimized parameters included: the algorithm parameter was configured to 'auto' to enable automatic selection of the algorithm, leaf_size was established at 30, the metric was designated as Minkowski with p=2 to denote Euclidean distance, and n_neighbors was fixed at 5 with weights set to uniform. However, in some of the cases the XGBoost classifier performed well, such as Dataset-2, unknown class classification while monitoring the USB port. The random forest classifier performed better than the other classifiers when working with Dataset-1 and for regular classification (without considering the possibility of unknown class) while monitoring the USB port.

Classification	Dataset	HDMI		USB	
		Accuracy	F1 Score	Accuracy	F1 Score
Regular	Dataset-1	87.36%	0.8747	71.35%	0.7063
	Dataset-2	94.25%	0.9415	56.77%	0.5661
Unknown	Dataset-1	86.25%	0.8638	98.34%	0.9814
	Dataset-2	93.75%	0.9364	98.85%	0.9885

Table 3: Base CNNs classification test accuracy(Config 1).

Classification	Dataset	HDMI		USB	
		Accuracy	F1 Score	Accuracy	F1 Score
Regular	Dataset-1	68.34%	0.6829	68.48%	0.6833
	Dataset-2	76.45%	0.7667	72.10%	0.7187
Unknown	Dataset-1	62.58%	0.6256	60.50%	0.6014
	Dataset-2	74.39%	0.7437	71.39%	0.7321

Table 4: CNN family classification test accuracy(Config 2).

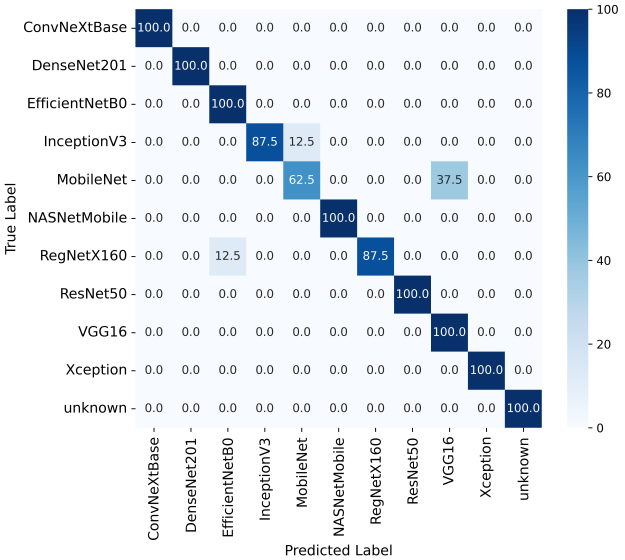


Figure 10: Confusion Matrix for the Classification of Base CNN models (Configuration 1), for the Dataset-2.

6.2.2 Classification results for video rendering inference experiment. For the video identification task, we again perform a multiclass classification. We use the same machine learning pipelines discussed earlier in this classification task. In this experiment, the K neighbor classifier with the normalizer scalar also performed the best.

The test accuracy obtained in this classification task with the USB port is 92.86%. The F1 score is 0.92. The scikit learn pipeline optimizes the classification task by setting the hyperparameters of the classifiers. In this experiment, the hyper parameters of the K neighbor classifier were set as follows. The leaf_size is set to 30, metric as Minkowski with p = 2 representing the Euclidean distance, and n_neighbors set to 5 with weights as uniform. We also present a confusion matrix for this experiment in Figure 11. We can observe that video 2 and video 4 were misclassified among themselves for a few test cases. On the other hand, with the identical classifier configuration, the test accuracy for the HDMI port achieved an impeccable 100%. When we considered the possibility of having an unknown class by setting the confidence threshold to 0.5 and providing unknown power leakage data, the best performing average test accuracy was observed from Dataset-2 on both HDMI and USB ports, which were 74.39% and 71.39% respectively. The sklearn classification configuration remained same as before in this case.

Classification	USB		HDMI	
	Accuracy	F1 Score	Accuracy	F1 Score
Regular	92.86%	0.92	100%	1
Unknown	96.00%	0.9589	96.43%	0.9609

Table 5: Video identification classification results

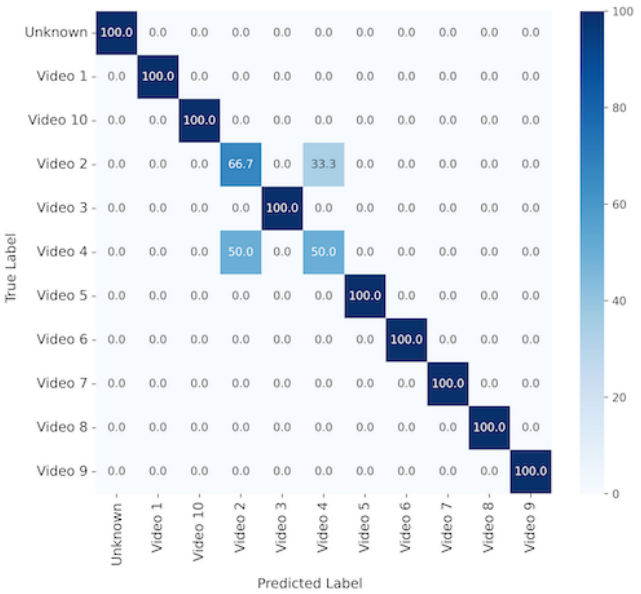


Figure 11: Confusion Matrix for the video rendering inference experiment.

7 Limitations

This section highlights two critical limitations of power side-channel attacks: the need for the attacker to be in close proximity to the victim (Section 7.1) and the challenge posed by concurrent program execution, which can render the attack infeasible (Section 7.2). Addressing background noise is thus essential.

7.1 Attacker Proximity

Our attack necessitates the attacker to meticulously read the current readings from the I/O port of the victim. This can be accomplished through either a USB cable directly linking the attack device or via wireless communication. Our sophisticated attack device can be extended to utilize a WiFi or Bluetooth module, replacing the wired USB connection between the victim and the attacker. This can help to minimize the risk of detection. However, the maximum feasible distance is constrained by the coverage range of Bluetooth or WiFi technology. Notably, certain Bluetooth 5.3 modules can have an effective range of 25 feet and, WiFi modules can span distances of up to 120 feet. Consequently, the attacker must be in close proximity to the victim to gather power side-channel data, necessitating a certain degree of nearness to the victim. On another note, the attacker could modify the attack device to store the power side channel data from the victim onto a temporary storage medium, such as an SD card. This allows the attacker to later retrieve the SD card from the compromised device connected to the victim's machine. In such a scenario, the attacker must be physically present near the victim's machine to connect the attack device initially and to retrieve the collected data. This can increase the risk of exposure of the attack.

7.2 Background Noise Consideration

While conducting our experiments, we deliberately chose not to disable any system processes. We ensured that all operating system-related processes remained active. To maintain a controlled environment, we meticulously avoided any additional services or software from the user end, such as web browsers, games, etc. However, theoretically speaking, the presence of other running processes on the system could potentially interfere with or distort the attack. We rigorously tested several fundamental operations, including matrix multiplication, video rendering, and CNN execution. If a similar process were to run concurrently, it could indeed weaken the attack. Consequently, background noise can significantly impact the attack.

8 Conclusion

This paper presents a novel power side-channel attack that leverages fine-grained power fluctuations in peripheral components — specifically USB and HDMI ports—instead of directly measuring processing units. Through controlled GPU experiments, we demonstrate that these peripheral power traces leak detailed information about operations such as matrix multiplication, video rendering, and CNN model architecture, enabling us to identify both the family and variant of deployed CNNs—and even discover previously unseen classes. By shifting focus to peripheral I/O, our findings broaden the scope of power side-channel threats and underscore the urgent need for security measures that protect the entire power-supply network, positioning peripheral ports as critical, previously overlooked attack vectors.

The power leakages enabling our attack could be due to several reasons. First, computer power supply units (PSU) stabilize the electrical current of system components using capacitors, MOSFETs and voltage regulation modules (VRMs). These components ensure consistent current flow, protecting against performance or power fluctuations [4]. However, due to wear and tear the effectiveness of power supply stabilization may deteriorate over time. This could contribute to the power draw patterns that our attack picks up at the USB and HDMI ports. Another possible reason could be that the power stabilization mechanisms of today's computers are simply not fine-grained enough to obfuscate the minute power fluctuation patterns that our algorithms leverage to drive the attacks. A thorough exploration on the potential contributions of these variables is part of our future work. The implications of this study offer new insights and opportunities for both attackers and defenders in the ongoing battle to secure computational devices.

References

[1] [n. d.]. Arduino MKR Zero. url: https://docs.arduino.cc/hardware/mkr-zero/. Accessed: January 2, 2024.
[2] [n. d.]. Nvidia Management Library. https://developer.nvidia.com/nvidia-management-library-nvml/. Accessed: June 2, 2024.
[3] [n. d.]. Nvidia system management interface. https://developer.nvidia.com/nvidia-system-management-interface/. Accessed: June 2, 2024.
[4] 2011. About VRMs & MOSFETs / Motherboard Safety with high-TDP processors. https://www.overclock.net/threads/about-vrms-mosfets-motherboard-safety-with-high-tdp-processors.1772883/. Accessed: April 4, 2024.
[5] 2023. Power Supply Connectors - How to Connect PC Power Connectors. https://pcfastlane.com/power-supply-connectors-guide-understanding-types-and-functions/. Accessed: April 4, 2024.
[6] 2024. TensorFlow module: tf.keras.applications. Website. https://www.tensorflow.org/api_docs/python/tf/keras/applications TensorFlow version: v2.12.

[7] Sraddhanjali Acharya, Abdul Serwadda, and Argenis V. Bilbao. 2021. That phone charging hub knows your video playlist!. In *2021 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/IOP/SCI)*. 160–169. https://doi.org/10.1109/SWC50871.2021.00031

[8] Brian Anderson and Barbara Anderson. 2010. *Seven deadliest USB attacks*. Syngress.

[9] Sayed Erfan Arefin and Abdul Serwadda. 2021. Deep Neural Exposure: You Can Run, But Not Hide Your Neural Network Architecture! *(IH&MMSec '21)*. Association for Computing Machinery, New York, NY, USA, 75–80. https://doi.org/10.1145/3437880.3460415

[10] Sayed Erfan Arefin and Abdul Serwadda. 2024. Dissecting the Origins of the Power Side-Channel in Deep Neural Networks. In *2024 IEEE 5th Annual World AI and IoT Congress*.

[11] Gary Bradski, Adrian Kaehler, et al. 2023. OpenCV: Open Source Computer Vision Library. Software available from http://opencv.org. Accessed: March 20, 2024.

[12] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (10 2001), 5–32. https://doi.org/10.1023/A:1010933404324

[13] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) *(KDD '16)*. Association for Computing Machinery, New York, NY, USA, 785–794. https://doi.org/10.1145/2939672.2939785

[14] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 248–255. https://doi.org/10.1109/CVPR.2009.5206848

[15] Debopriya Roy Dipta and Berk Gulmezoglu. 2022. DF-SCA: Dynamic Frequency Side Channel Attacks are Practical. In *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC '22)*. Association for Computing Machinery, New York, NY, USA, 841–853. https://doi.org/10.1145/3564625.3567979

[16] Neil Fox. 2022. How to Use Autoruns to Detect and Remove Malware on Windows. Varonis. https://www.varonis.com/blog/how-to-use-autoruns

[17] Hasindu Gamaarachchi and Harsha Ganegoda. 2018. Power Analysis Based Side Channel Attack. arXiv:1801.00932 [cs.CR]

[18] Glmark2 Development Team. 2024. glmark2. https://github.com/glmark2/glmark2. Accessed: March 20, 2024.

[19] Gongde Guo, Hui Wang, David Bell, Yaxin Bi, and Kieran Greer. 2003. KNN Model-Based Approach in Classification. In *On The Move to Meaningful Internet Systems 2003: CoopIS, DOA, and ODBASE*, Robert Meersman, Zahir Tari, and Douglas C. Schmidt (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 986–996.

[20] Charles R Harris, K Jarrod Millman, Stéfan J van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. 2020. Array programming with NumPy. *Nature* 585, 7825 (2020), 357–362.

[21] M.A. Hearst, S.T. Dumais, E. Osuna, J. Platt, and B. Scholkopf. 1998. Support vector machines. *IEEE Intelligent Systems and their Applications* 13, 4 (1998), 18–28. https://doi.org/10.1109/5254.708428

[22] Hanjae Jeong, Younsung Choi, Woongryel Jeon, Fei Yang, Yunho Lee, Seungjoo Kim, and Dongho Won. 2007. Vulnerability analysis of secure USB flash drives. In *2007 IEEE International Workshop on Memory Technology, Design and Testing*. 61–64. https://doi.org/10.1109/MTDT.2007.4547620

[23] Nandan Kumar Jha, Sparsh Mittal, Binod Kumar, and Govardhan Mattela. 2020. DeepPeep: Exploiting Design Ramifications to Decipher the Architecture of Compact DNNs. *J. Emerg. Technol. Comput. Syst.* 17, 1, Article 5 (oct 2020), 25 pages. https://doi.org/10.1145/3414552

[24] Kaspersky. [n. d.]. Equation Group: Questions and Answers. 2015. url: https://securelist. com/files/2015/02. *Equation _ group_questions_and_answers. pdf* ([n. d.]).

[25] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: a highly efficient gradient boosting decision tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) *(NIPS'17)*. Curran Associates Inc., Red Hook, NY, USA, 3149–3157.

[26] Alexander S. La Cour, Khurram K. Afridi, and G. Edward Suh. 2021. Wireless Charging Power Side-Channel Attacks. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) *(CCS '21)*. Association for Computing Machinery, New York, NY, USA, 651–665. https://doi.org/10.1145/3460120.3484733

[27] Chavy Levi. 2020. Guide to Computer Ports and Connectors: USB, Thunderbolt 3, HDMI. Hardware Corner. https://www.hardware-corner.net/guides/guide-to-computer-ports-and-connectors

[28] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. 2021. PLATYPUS: Software-based Power Side-Channel Attacks on x86. In *2021 IEEE Symposium on Security and Privacy (SP)*. 355–371. https://doi.org/10.1109/SP40001.2021.00063

[29] Chen Liu, Abhishek Chakraborty, Nikhil Chawla, and Neer Roggel. 2022. Frequency Throttling Side-Channel Attack. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. ACM. https://doi.org/10.1145/3548606.3560682

[30] Chao Luo, Yunsi Fei, Pei Luo, Saoni Mukherjee, and David Kaeli. 2015. Side-channel power analysis of a GPU AES implementation. In *2015 33rd IEEE International Conference on Computer Design (ICCD)*. 281–288. https://doi.org/10.1109/ICCD.2015.7357115

[31] Brian McFee, Colin Raffel, Dawen Liang, Daniel PW Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. 2015. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th python in science conference*. 18–25.

[32] Karsten Nohl, Sascha Krißler, and Jakob Lell. 2014. BadUSB—On Accessories That Turn Evil. In *BlackHat USA 2014*. Security Research Labs. https://www.blackhat.com/us-14/briefings.html#Nohl

[33] NVTop Contributors. 2024. NVTop: NVIDIA GPUs Monitoring Tool. https://github.com/Syllo/nvtop. Accessed: March 20, 2024.

[34] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. 2011. Scikit-learn: Machine learning in Python. *Journal of machine learning research* 12, Oct (2011), 2825–2830.

[35] Dung Vu Pham, Ali Syed, and Malka N. Halgamuge. 2011. Universal serial bus based software attacks and protection solutions. *Digital Investigation* 7, 3 (2011), 172–184. https://doi.org/10.1016/j.diin.2011.02.001

[36] Avram Piltch. 2022. A Guide to Computer Ports and Adapters. Laptop Mag. https://www.laptopmag.com/articles/port-and-adapter-guide

[37] Annisa Dwiayu Ramadhanty, Avon Budiono, and Ahmad Almaarif. 2020. Implementation and Analysis of Keyboard Injection Attack using USB Devices in Windows Operating System. In *2020 3rd International Conference on Computer and Informatics Engineering (IC2IE)*. 449–454. https://doi.org/10.1109/IC2IE50715.2020.9274631

[38] Majid Sabbagh, Yunsi Fei, and David Kaeli. 2020. A Novel GPU Overdrive Fault Attack. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC18072.2020.9218690

[39] Kittipong Satrinekarn. 2023. Black 850W Power Supply Unit with Cables and Switch I/O for ATX Tower PC Cases. https://www.vecteezy.com/photo/2221759-black-850w-power-supply-unit-with-cables-and-switch-i-o-for-atx-tower-pc-cases-isolated-on-a-white-background. Accessed: April 8, 2024.

[40] Ron Segev and Avi Mendelson. 2023. The Use of Performance-Countersto Perform Side-Channel Attacks. In *Cyber Security, Cryptology, and Machine Learning: 7th International Symposium, CSCML 2023, Be'er Sheva, Israel, June 29–30, 2023, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 216–233. https://doi.org/10.1007/978-3-031-34671-2_16

[41] Nader Sehatbakhsh, Baki Berkay Yilmaz, Alenka Zajic, and Milos Prvulovic. 2020. A New Side-Channel Vulnerability on Modern Computers by Exploiting Electromagnetic Emanations from the Power Management Unit. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 123–138. https://doi.org/10.1109/HPCA47549.2020.00020

[42] SparkFun Electronics. 2023. SparkFun Qwiic Shield for Arduino Nano. https://www.sparkfun.com/products/14544. Accessed: June 2, 2024.

[43] Usama Tariq, Imran Ahmed, Ali Kashif Bashir, and Kamran Shaukat. 2023. A Critical Cybersecurity Analysis and Future Research Directions for the Internet of Things: A Comprehensive Review. *Sensors (Basel)* 23, 8 (19 Apr 2023), 4117. https://doi.org/10.3390/s23084117

[44] Seiya Tokui et al. 2023. CuPy: A NumPy-compatible array library for GPU-accelerated computing with Python. https://cupy.dev/. Accessed: March 21, 2024.

[45] Yingchen Wang, Riccardo Paccagnella, Elizabeth Tang He, Hovav Shacham, Christopher W. Fletcher, and David Kohlbrenner. 2023. Hertzbleed: Turning Power Side-Channel Attacks Into Remote Timing Attacks on x86. *IEEE Micro* 43, 4 (2023), 19–27. https://doi.org/10.1109/MM.2023.3274619

[46] Qing Yang, Paolo Gasti, Gang Zhou, Aydin Farajidavar, and Kiran S. Balagani. 2017. On Inferring Browsing Activity on Smartphones via USB Power Analysis Side-Channel. *IEEE Transactions on Information Forensics and Security* 12, 5 (2017), 1056–1066. https://doi.org/10.1109/TIFS.2016.2639446

[47] Xiang Zhang, Aidong Adam Ding, and Yunsi Fei. 2023. Deep-Learning Model Extraction Through Software-Based Power Side-Channel. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. 1–9. https://doi.org/10.1109/ICCAD57390.2023.10323806

[48] Mark Zhao and G. Edward Suh. 2018. FPGA-Based Remote Power Side-Channel Attacks. In *2018 IEEE Symposium on Security and Privacy (SP)*. 229–244. https://doi.org/10.1109/SP.2018.00049